

Source paper: Shanto Rahman, Saikat Dutta, and August Shi. 2025. Understanding and Improving Flaky Test Classification. Proc. ACM Program. Lang. 9, OOPSLA2, Article 320 (October 2025), 27 pages. <https://doi.org/10.1145/3763098>

I reproduced FlakyLens's RQ1 first because it's the most self-contained piece of the paper: a fixed six-category label set and project-wise four fold cross-validation. GPU time was the limiting factor this round, so I focused on getting RQ1 right and used the reproduction to identify potential directions for further research.

Experimental Combinations

1. No SMOTE, seed 42. Followed the paper's written methodology using class-balanced focal loss only.
2. SMOTE on, seed 14. Followed the released implementation with SMOTE and a fixed split seed selected for better fold balance.
3. Evaluated the provided checkpoints on their corresponding project splits.

Why SMOTE Was Off

The paper's methodology (4.2.2) only mentions class-balanced weights and focal loss for handling the class imbalance, no oversampling was mentioned. SMOTE shows up only in the released code, not the paper text, so I ran without it first to get a clean baseline that matches what's actually written before adding anything the paper doesn't describe.

When I looked at how SMOTE is implemented there, I noticed that a TF-IDF vector can't be turned back into real Java code, every synthetic row it generates gets the same placeholder text standing in for the code, so I went in expecting it might not help much. In practice it did increase macro-F1 from 56.86% to 63.10%, mostly from Test Order Dependency and Unordered Collections recovering a lot of ground. That said, I changed the split seed at the same time I turned SMOTE on, so I can't yet say how much of that gain is SMOTE and how much is the better-balanced split.

Why Seed 14

The project split in the released code (`create_train_test_groups`) shuffles projects using Python's global random module, and it's never seeded anywhere in the file. So the authors' own Table 2 split can't be recovered. On top of that, the split only guarantees a floor of at least 4 test examples per category, with no upper limit, so a rare category can end up almost entirely in a single fold. Under seed 42, Test Order Dependency (93 examples total) had 79 of them, 85%, sitting in just one fold's test set. I scanned 50 seeds to see if this was fixable. 49 out of 50 valid splits still concentrated 40% to 85% of Test Order Dependency into a single fold. Among the 50 seeds I examined, seed 14 produced the lowest maximum TOD concentration, at 46%.

I also considered adding an upper bound to balance the folds, assigning projects from the rarest categories to the fold with the lowest current count. However, this was only a proposed approach and I did not implement or test it. The idea was motivated by the possibility that some folds may contain a disproportionately large share of rare-category projects, which could affect the evaluation results.

Seed	Fold 1	Fold 2	Fold 3	Fold 4	TOD distribution
42 (initial)	5	79	6	3	85%
14 (chosen)	9	32	43	9	46%

Why the Authors' Provided Weights Scored Too High

I evaluated the four separately provided checkpoints against their matching splits and obtained a pooled macro-F1 of 89.13%, which was unexpectedly high compared with the paper's reported 65.79%. I then tested each checkpoint on all four folds. The checkpoints did not consistently perform best on their corresponding fold. For example, checkpoint 1 scored 78% on its own fold but 83%–94% on the others. This suggests that the provided checkpoints and split files were not generated from fully fold-isolated training. Since the exact training process is unavailable, I excluded this run from the final results.

Why My Numbers Differ Slightly From the Paper

- **Different project splits:** My split and the paper's split contain different held-out projects, so the results can vary across seeds.
- **Small categories are more sensitive:** Categories such as Concurrency and Test Order Dependency have fewer samples, so their scores can change more depending on which projects are held out.
- **Pipeline consistency:** Despite differences in the overall F1, several category-level results and the Concurrency → Async Wait confusion pattern were close to the paper, particularly in the non-SMOTE baseline. This provides some evidence that the implementation is behaving consistently, while the remaining differences may be due to the different project splits and experimental settings.

Limitation

- **Unbalanced project-wise splits:** I found that different project splits can distribute the rare flaky categories quite differently across folds, which can affect the final results.
- **Limited flaky categories:** The dataset covers only five flaky categories, so other real-world causes such as network, environment, GUI, or resource-related flakiness are not considered.
- **Single-label classification:** Each test has only one flaky label, although a test may have multiple interacting causes.
- **Source-code-only analysis:** The model mainly looks at the test code, but some flaky behaviour may depend on runtime conditions, execution history, or the environment.
- **Limited LLM comparison:** The LLM comparison is mainly zero-shot, so it does not show how stronger models might perform when they are adapted or fine-tuned for flaky-test classification.

Potential Research Contribution

Based on the reproduction and the limitations I identified, I am interested in exploring reliable and robust software testing using AI/LLMs. My goal is not to focus only on improving FlakyLens but to use the insights from this work as a starting point for broader research in reliable AI-assisted software engineering. Possible directions include:

- Develop more balanced fold construction and benchmarking methods using relevance ranking to limit rare-category concentration and improve classifier generalization to unseen projects.
- Explore multi-source approaches by combining test code with execution history, runtime behaviour, logs, and other software artifacts to better understand and diagnose flaky tests.
- Explore how LLMs can better understand program behaviour and use that understanding for test generation, analysis, debugging, and flaky-test diagnosis.
- Use LLMs or other generative methods to create realistic synthetic examples for rare categories, reducing reliance on oversampling methods that may produce unrealistic or label-inconsistent data.
- Explore multi-label classification to capture cases where a test can have multiple interacting flaky causes.